

Advanced Programming

Assignment 9 | Spring 2026

Name	Gaurav Nath
Roll No.	CSB24073
Subject	Advanced Programming
Assignment	Assignment No. 9
Semester	Spring 2026

Banking System — OOP in Java

Question

Design a banking system in Java with:

- A base class Account containing private fields: accountNumber, ownerName, balance
- Provide getters/setters and at least two constructors (use constructor chaining)
- Implement deposit() and withdraw() with proper validation
- Add a display() method
- Extend with SavingsAccount — add interestRate, override display() and show interest
- Extend with CurrentAccount — add overdraftLimit, restrict withdrawals accordingly

Implementation must clearly show:

- Proper encapsulation (no direct field access)
- Constructor overloading and chaining using this(...)
- Inheritance and method overriding using @Override and super
- Polymorphism — store objects in an Account reference list and call display()
- Basic validation using exceptions for invalid operations

Implementation

File 1: Account.java (Base Class)

```
public class Account {  
  
    private String accountNumber;  
    private String ownerName;  
    private double balance;  
  
    // Constructor 1 — chains to Constructor 2 with default balance 0.0  
    public Account(String accountNumber, String ownerName) {  
        this(accountNumber, ownerName, 0.0);  
    }  
}
```

```

    }

    // Constructor 2 - full constructor, validates balance before assigning
    public Account(String accountNumber, String ownerName, double balance) {
        if (balance < 0)
            throw new IllegalArgumentException("Opening balance cannot be
negative.");
        this.accountNumber = accountNumber;
        this.ownerName      = ownerName;
        this.balance        = balance;
    }

    // Getters - only way to read private fields from outside
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName()     { return ownerName; }
    public double getBalance()       { return balance; }

    // Setter for ownerName with validation
    public void setOwnerName(String ownerName) {
        if (ownerName == null || ownerName.isBlank())
            throw new IllegalArgumentException("Owner name cannot be empty.");
        this.ownerName = ownerName;
    }

    // Protected - only subclasses can update balance directly
    protected void setBalance(double balance) { this.balance = balance; }

    public void deposit(double amount) {
        if (amount <= 0)
            throw new IllegalArgumentException("Deposit amount must be
positive.");
        balance += amount;
        System.out.println("Deposited Rs." + amount + " | New balance: Rs." +
balance);
    }

    public void withdraw(double amount) {
        if (amount <= 0)
            throw new IllegalArgumentException("Withdrawal amount must be
positive.");
        if (amount > balance)
            throw new IllegalStateException("Insufficient funds. Balance: Rs."
+ balance);
        balance -= amount;
        System.out.println("Withdrawn Rs." + amount + " | New balance: Rs." +
balance);
    }

    public void display() {
        System.out.println("-----");
        System.out.println("Account No : " + accountNumber);
        System.out.println("Owner      : " + ownerName);
        System.out.println("Balance    : Rs." + balance);
    }
}

```

File 2: SavingsAccount.java (Subclass)

```

public class SavingsAccount extends Account {

```

```

private double interestRate;

// Constructor 1 - defaults to 0 balance, 4% interest
public SavingsAccount(String accountNumber, String ownerName) {
    this(accountNumber, ownerName, 0.0, 0.04);
}

// Constructor 2 - full constructor
public SavingsAccount(String accountNumber, String ownerName,
    double balance, double interestRate) {
    super(accountNumber, ownerName, balance);
    if (interestRate < 0 || interestRate > 1)
        throw new IllegalArgumentException("Rate must be between 0 and
1.");
    this.interestRate = interestRate;
}

public double getInterestRate() { return interestRate; }

public void setInterestRate(double rate) {
    if (rate < 0 || rate > 1)
        throw new IllegalArgumentException("Rate must be between 0 and
1.");
    this.interestRate = rate;
}

public double calculateInterest() { return getBalance() * interestRate; }

@Override
public void display() {
    super.display(); // reuse Account's display output
    System.out.println("Type      : Savings Account");
    System.out.println("Int. Rate  : " + (interestRate * 100) + "%");
    System.out.println("Yearly Int.: Rs." + calculateInterest());
    System.out.println("-----");
}
}

```

File 3: CurrentAccount.java (Subclass)

```

public class CurrentAccount extends Account {

    private double overdraftLimit;

    // Constructor 1 - defaults to 0 balance, Rs.10,000 overdraft
    public CurrentAccount(String accountNumber, String ownerName) {
        this(accountNumber, ownerName, 0.0, 10000.0);
    }

    // Constructor 2 - full constructor
    public CurrentAccount(String accountNumber, String ownerName,
        double balance, double overdraftLimit) {
        super(accountNumber, ownerName, balance);
        if (overdraftLimit < 0)
            throw new IllegalArgumentException("Overdraft limit cannot be
negative.");
        this.overdraftLimit = overdraftLimit;
    }

    public double getOverdraftLimit() { return overdraftLimit; }
}

```

```

        // Overrides withdraw – allows balance to go negative up to overdraftLimit
        @Override
        public void withdraw(double amount) {
            if (amount <= 0)
                throw new IllegalArgumentException("Withdrawal amount must be
positive.");
            if (amount > getBalance() + overdraftLimit)
                throw new IllegalStateException(
                    "Exceeds overdraft limit. Max: Rs." + (getBalance() +
overdraftLimit));
            setBalance(getBalance() - amount);
            System.out.println("Withdrawn Rs." + amount + " | New balance: Rs." +
getBalance());
        }

        @Override
        public void display() {
            super.display();
            System.out.println("Type          : Current Account");
            System.out.println("Overdraft   : Rs." + overdraftLimit);
            System.out.println("-----");
        }
    }
}

```

File 4: Main.java (Driver — Polymorphism Demo)

```

import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {

        // Constructor overloading – creating with and without opening balance
        SavingsAccount s1 = new SavingsAccount("SAV001", "Gaurav Nath", 5000.0,
0.06);
        SavingsAccount s2 = new SavingsAccount("SAV002", "Riya Sen");
        CurrentAccount c1 = new CurrentAccount("CUR001", "Tezpur Traders",
2000.0, 15000.0);

        s1.deposit(3000.0);
        s1.withdraw(1000.0);
        c1.withdraw(15000.0);    // goes into overdraft – allowed

        // POLYMORPHISM – Account reference holds both Savings and Current
objects
        List<Account> accounts = new ArrayList<>();
        accounts.add(s1);
        accounts.add(s2);
        accounts.add(c1);

        System.out.println("\n===== ALL ACCOUNTS =====");
        for (Account acc : accounts) {
            acc.display();    // Java picks correct display() at runtime
        }

        // Exception handling demo
        try {
            s2.withdraw(999999.0);
        } catch (IllegalStateException e) {

```

```

        System.out.println("Caught: " + e.getMessage());
    }
    try {
        s1.deposit(-500.0);
    } catch (IllegalArgumentException e) {
        System.out.println("Caught: " + e.getMessage());
    }
}
}
}

```

OOP Concepts Demonstrated

Concept	Where	How
Encapsulation	Account.java	All fields private; accessed only via getters/setters
Constructor Chaining	Account, Savings, Current	this(...) delegates to the full constructor
Inheritance	SavingsAccount, Current	extends Account; reuses all base fields and methods
Method Overriding	@Override display()	Each subclass adds its own fields to the display output
Polymorphism	List<Account> in Main	acc.display() calls the correct subclass version at runtime
Exception Validation	deposit(), withdraw()	IllegalArgumentException and IllegalStateException used

Compile & Run

```

cd BankingSystem
javac Account.java SavingsAccount.java CurrentAccount.java Main.java
java Main

```

Output Screenshots

Terminal output:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Downloads D Folder\Advanced Programming 2\BankingSystem> Rename-Item savingsAccount.java
SavingsAccount.java
PS D:\Downloads D Folder\Advanced Programming 2\BankingSystem> javac *.java
>> java Main
Deposited Rs.3000.0 | New balance: Rs.8000.0
Withdrawn Rs.1000.0 | New balance: Rs.7000.0
Withdrawn Rs.15000.0 | New balance: Rs.-13000.0

===== ALL ACCOUNTS =====
-----
Account No : SAV001
Owner      : Gaurav Nath
Balance    : Rs.7000.0
Type       : Savings Account
Int. Rate  : 6.0%
Yearly Int.: Rs.420.0
-----
Account No : SAV002
Owner      : Riya Sen
Balance    : Rs.0.0
Type       : Savings Account
Int. Rate  : 4.0%
Yearly Int.: Rs.0.0
-----
Account No : CUR001
Owner      : Tezpur Traders
Balance    : Rs.-13000.0
Type       : Current Account
Overdraft  : Rs.15000.0
-----

--- Exception Demo ---
Caught exception: Insufficient funds. Balance: Rs.0.0
Caught exception: Deposit amount must be positive.
❖ PS D:\Downloads D Folder\Advanced Programming 2\BankingSystem>
```

VS Code file structure:

